«Клієнт-серверна архітектура додатку. Метод комунікації: поштові скриньки»

Мета роботи

Набуття практичних умінь у створенні програмних додатків за архітектурою «клієнт-сервер» із використанням поштової скриньки (mailslot) для обміну даними.

Завдання роботи

Розробити програмний комплекс, що включає:

- додаток із функціоналом сервера;
- два додатки із функціоналом клієнтів.

Функції програми-сервера:

- створення поштової скриньки;
- читання поштової скриньки з певним інтервалом часу;
- прийом даних від клієнтських додатків;
- відображення отриманих даних на екрані;
- закриття поштової скриньки.

Функції програми-клієнта:

- відкриття поштової скриньки;
- підготовка та відправка даних на сервер.

Варіант для виконання лабораторної роботи розрахувати за формулою:

9 – «варіант виконання лабораторної роботи «Клієнт-серверна архітектура додатку. Метод комунікації: сокети» »

З таблиці №1 лабораторної роботи «Клієнт-серверна архітектура додатку. Метод комунікації: сокети» взяти завдання за розрахованим варіантом.

Завдання «з зірочкою»: Клієнт 3 – Надсилання результатів математичних обчислень (наприклад, площа прямокутника) за довільними початковими даними на сервер.

Теоретичні відомості

У сучасних операційних системах паралельне виконання процесів вимагає не лише синхронізації їхньої роботи (узгодження за допомогою сигналів чи блокувань), а й обміну повідомленнями довільного вмісту.

При цьому виникає проблема: процес-відправник і процес-отримувач можуть працювати в різному ритмі. Відправлення повідомлення та його отримання майже ніколи не збігаються за часом.

Щоб забезпечити коректний обмін даними, вводиться механізм тимчасового зберігання повідомлень у спеціальному буфері, який називається поштова скринька (mailslot).

Поштова скринька (Mailslots) – це механізм міжпроцесної та мережевої комунікації, який дозволяє організувати односпрямований обмін повідомленнями між клієнтом і сервером.

Процес, який створює поштову скриньку, виступає сервером (Mailslot Server). Він очікує повідомлення та зчитує їх у міру надходження.

Інші процеси, які хочуть передати дані серверу, виступають клієнтами (Mailslot Clients). Вони записують повідомлення в поштову скриньку, не чекаючи підтвердження прийому.

Особливості роботи:

- усі вхідні повідомлення зберігаються у скриньці до моменту, коли сервер їх прочитає;
- повідомлення завжди додаються у кінець черги (FIFO – перший прийшов, перший оброблений);
- клієнт може надсилати повідомлення:
- на скриньку, створену на тому ж комп'ютері;
- на скриньку іншого комп'ютера в мережі;
- або на всі скриньки з однаковим ім'ям у межах домену (broadcast у межах мережі).

У Windows API передбачені функції для роботи з поштовими скриньками, що можуть бути використані у Visual Studio 2015 та новіших версіях:

- CreateMailslot – створення поштової скриньки (сервером);
- WriteFile – запис повідомлення у поштову скриньку (клієнтом);
- ReadFile – читання повідомлення із поштової скриньки (сервером);
- GetMailslotInfo – отримання інформації про стан скриньки (кількість повідомлень, розмір черги тощо);
- CloseHandle – типове закриття відкритого дескриптора об'єкту.

Visual Studio надає можливість розробляти розподілені додатки для організації обміну даними:

- серверна частина створює поштову скриньку та чекає на повідомлення;
- клієнтська частина відкриває поштову скриньку та надсилає повідомлення.

Переваги:

- простота використання (зручний API Windows);
- підтримка односпрямованої комунікації між процесами як локально, так і по мережі;

– можливість широкомовної розсилки повідомлень у межах домену.

Недоліки:

– односпрямованість (клієнт → сервер), для двостороннього зв'язку потрібно створювати кілька скриньок;

– обмежені можливості у порівнянні з сокетами (немає гарантованої доставки, контроль розміру повідомлення покладається на програміста);

– використання здебільшого для простих мережних сервісів чи локальних комунікацій.

Приклад виконання

Клієнт 1 повинен підготувати і передати серверу такі дані: – ширину й висоту вікна додатку; – ширину й висоту екрану дисплея.	Клієнт 2 т повинен підготувати і передати серверу такі дані: – міжрядковий інтервал при виводі тексту; – ширину й висоту піктограми (іконки) додатку.

У додатку-серверу необхідно створити поштову скриньку та організувати стеження за її станом.

Для цього додаємо змінні:

```
// server includes and variables
static HWND hwndEdit;
char mess[2048];
char* m_mess = mess;
#include <stdlib.h>
#include <stdio.h>
#include <string>
HANDLE hMailslot;
LPSTR MailslotName = "\\\\.\\mailslot\\lab7";   //ім'я скриньки
DWORD cbRead;                                   //кількість прийнятих байтів даних
char szBuf[512];                                //буфер
#define IDT_TIMER 1001                          //таймер читача поштової скрині

// Global Variables:
...
```

У віконній функції додаємо обробник події *WM_CREATE* для створення текстового поля відображення статусу роботи поштового серверу:

```
       case WM_CREATE:
              hwndEdit = CreateWindow(TEXT("EDIT"), TEXT("Лабораторна робота - Поштова
скриня.\r\n"),
                     WS_CHILD | WS_VISIBLE | WS_VSCROLL | ES_LEFT | ES_MULTILINE |
ES_AUTOVSCROLL, 20, 10, 400, 460, hWnd, NULL, hInst, NULL);
              break;
```

В обробнику подій `case WM_COMMAND:... switch (wmId)` додаємо пару гілок, пов'язаних з пунктами меню (рис.1).

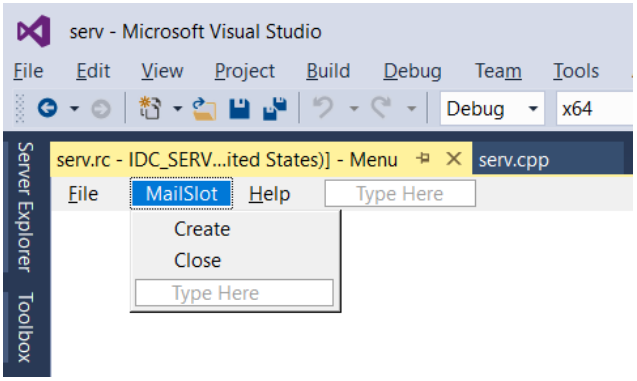


Рисунок 1 – Редагування головного меню додатку сервера

Створюємо поштову скриньку з дескриптором *hMailslot* використовуючи функцію *CreateMailslot*. У випадку виникнення помилки, виводимо її код і завершуємо роботу. Для стеження за поштовою скринькою вводимо таймер *IDT_TIMER* з періодом спрацьовування 1 с.

```
case 32771://ID_MAILSLOT_CREATE:
	sprintf_s(mess, "Поштова скриня\r\n");
	SendMessage(hwndEdit, WM_SETTEXT, 0, (LPARAM)m_mess);
	// створення та запуск поштової скрині
	hMailslot = CreateMailslot(MailslotName, 0, MAILSLOT_WAIT_FOREVER, NULL);
	if (hMailslot == INVALID_HANDLE_VALUE)
	{
		sprintf_s(mess, "%s \r\n Поштовий сервер. Error: %d \r\n ", m_mess,
GetLastError());
		SendMessage(hwndEdit, WM_SETTEXT, 0, (LPARAM)m_mess);
		break;
	}
	sprintf_s(mess, "%s\r\nСервер запущено\r\n", m_mess);
	SendMessage(hwndEdit, WM_SETTEXT, 0, (LPARAM)m_mess);
	SetTimer(hWnd, IDT_TIMER, 1000, (TIMERPROC)NULL); //запуск таймеру
	break;
```

Для закриття поштової скриньки слід використовувати типовий підхід по закриття дескриптора об’єкту:

```
case 32772://ID_MAILSLOT_CLOSE:
	CloseHandle(hMailslot);
	sprintf_s(mess, "%s\r\nПоштовий сервер закрито", m_mess);
	SendMessage(hwndEdit, WM_SETTEXT, 0, (LPARAM)m_mess);
	break;
```

Ну і саме головне – це організувати безпосереднє читання почтової скриньки за таймером. Для цього слід додати гілку обробника події спрацьовування таймера. Використовуючи функцію *GetMailslotInfo* стежимо за станом поштової скриньки. Якщо у каналі зв’язку є Mailslot-повідомлення (результат виконання попередньої функції відрізняється від нуля), то читаємо скриньку типовою функцією *ReadFile* та виводимо його вміст у текстове поле додатку сервера.

```
case WM_TIMER:
	switch (wParam)
```

```
	{
	case IDT_TIMER:
		DWORD cbMessage, cMessage, cbRead;
		BOOL fResult = GetMailslotInfo(hMailslot, NULL, &cbMessage, &cMessage, NULL);

		if (fResult && cMessage != 0)
			{
			ReadFile(hMailslot, szBuf, 512, &cbRead, NULL);
			sprintf_s(mess, "%s\r\n%s\r\n", m_mess, szBuf);
			SendMessage(hwndEdit, WM_SETTEXT, 0, (LPARAM)m_mess);
			}
		break;
	}
	break;
```

У додатку-клієнті необхідно з'єднатися з поштовою скринькою, використовуючи функцію створення файлу *CreateFile*. Запис у поштову скриньку проводиться аналогічно запису до стандартного файлу з використанням функції *WriteFile*. Перелік глобальних змінних клієнта майже такий як і у додатку сервера. Для реалізації цих дій достатньо реалізувати обробник зовнішньої події (відповідного пункту меню). Текст повідомлення зчитується з текстового поля клієнтського додатку:

```
case 32771:
	// формування повідомлення для відправлення
	cbWritten = SendMessageA(hwndEdit, WM_GETTEXTLENGTH, 0, 0);
	SendMessageA(hwndEdit, WM_GETTEXT, (WPARAM)cbWritten, (LPARAM)szBuf);
	// підключення до поштового серверу
	hMailslot = CreateFile(ServerName, GENERIC_READ | GENERIC_WRITE, FILE_SHARE_READ |
FILE_SHARE_WRITE,
	NULL, OPEN_EXISTING, 0, NULL);
	if (hMailslot == INVALID_HANDLE_VALUE)
	{
		sprintf_s(mess, "%s\r\nПомилка поштового серверу: %d\r\n", m_mess,
GetLastError());
		SendMessage(hwndEdit, WM_SETTEXT, 0, (LPARAM)m_mess);
		break;
	}
	// запис на поштовий сервер
	WriteFile(hMailslot, szBuf, strlen(szBuf) + 1, &cbWritten, NULL);
	break;
```

Формування повідомлення за замовченням можна пов’язати з обробником події WM_PAINT. Детально про визначення системних метрик розглядалося у Лабораторній роботі №1.

```
	// формування дефолтного повідомлення
	int minhi = GetSystemMetrics(SM_CXSCREEN);
	int minwi = GetSystemMetrics(SM_CYSCREEN);
	RECT R;
	GetWindowRect(hWnd, &R);
	LONG width = R.right - R.left;
	LONG height = R.bottom - R.top;
	sprintf_s(mess, "Дані Клієнта #1:\r\n - ширина екрану= % d \r\n - висота екрану= %
d\r\n - висота вікна = %i \r\n - ширина вікна = % i ",
		minhi, minwi, height, width);
	SendMessage(hwndEdit, WM_SETTEXT, 0, (LPARAM)m_mess);
```

Аналогічні дії слід виконати і для розробки другого клієнтського додатку.

В результаті виконання завдання маємо три додатки: сервер та два клієнти. На наступних рисунках продемонстровано працездатність додатків.

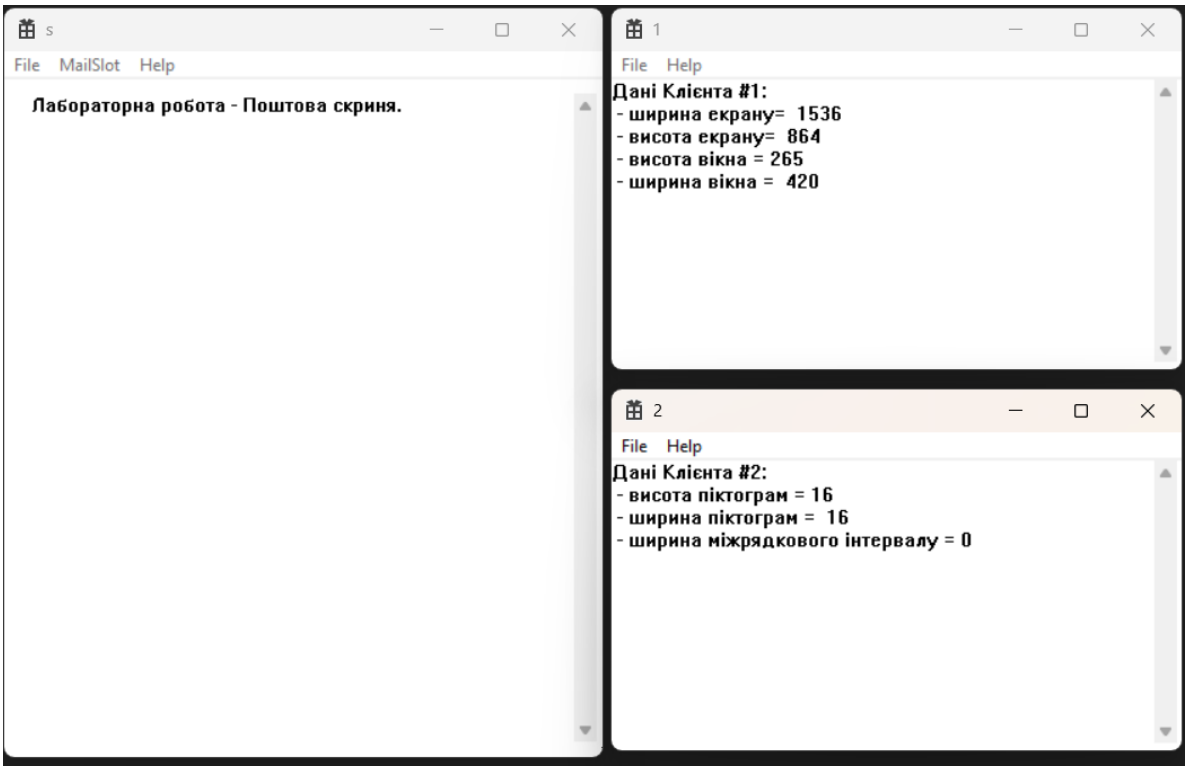


Рисунок 2 – Стартовий стан додатків

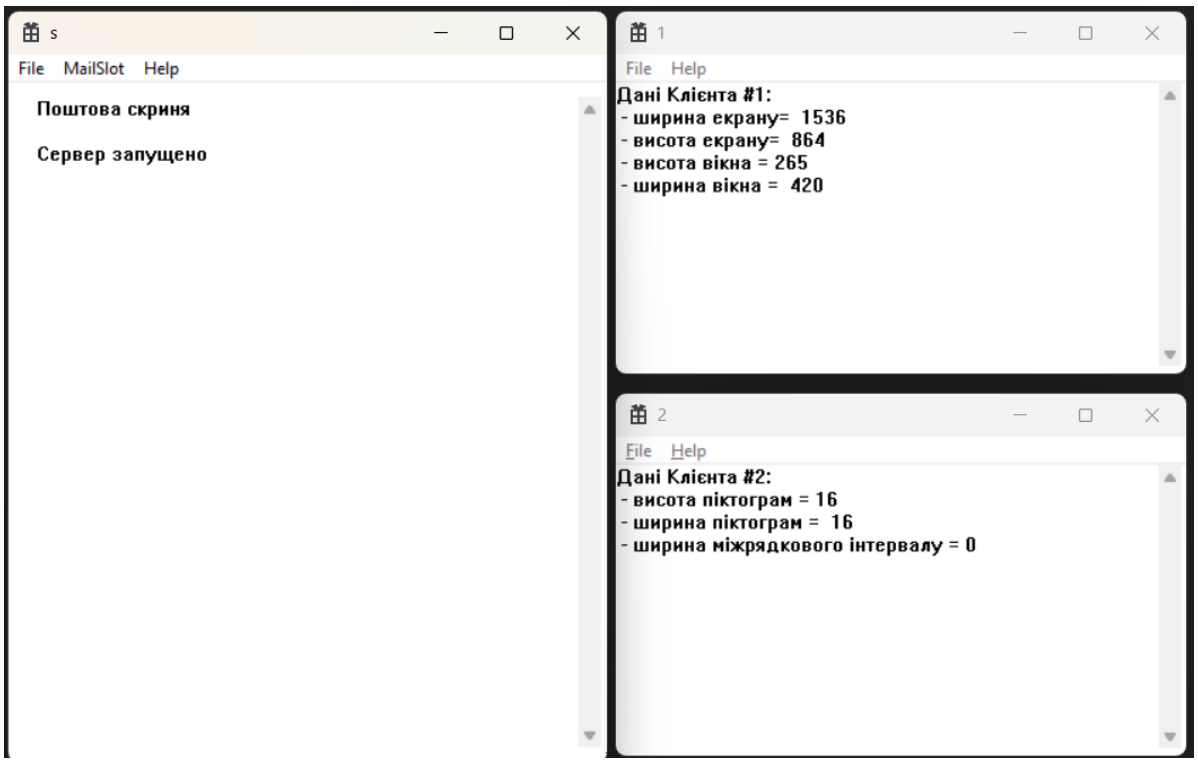


Рисунок 3 – Результат запуску поштового серверу

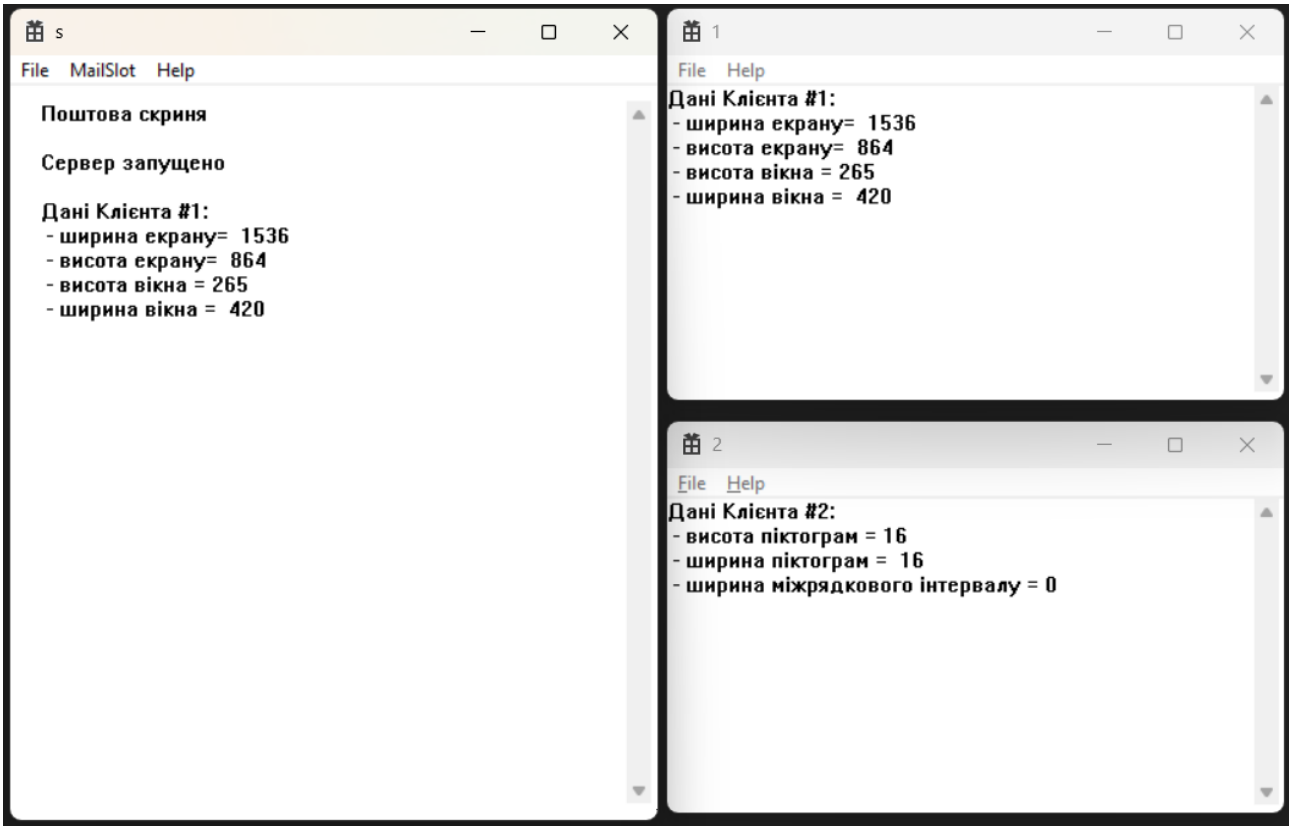


Рисунок 4 – Результат надсилання дефолтного повідомлення від «Клієнт №1»

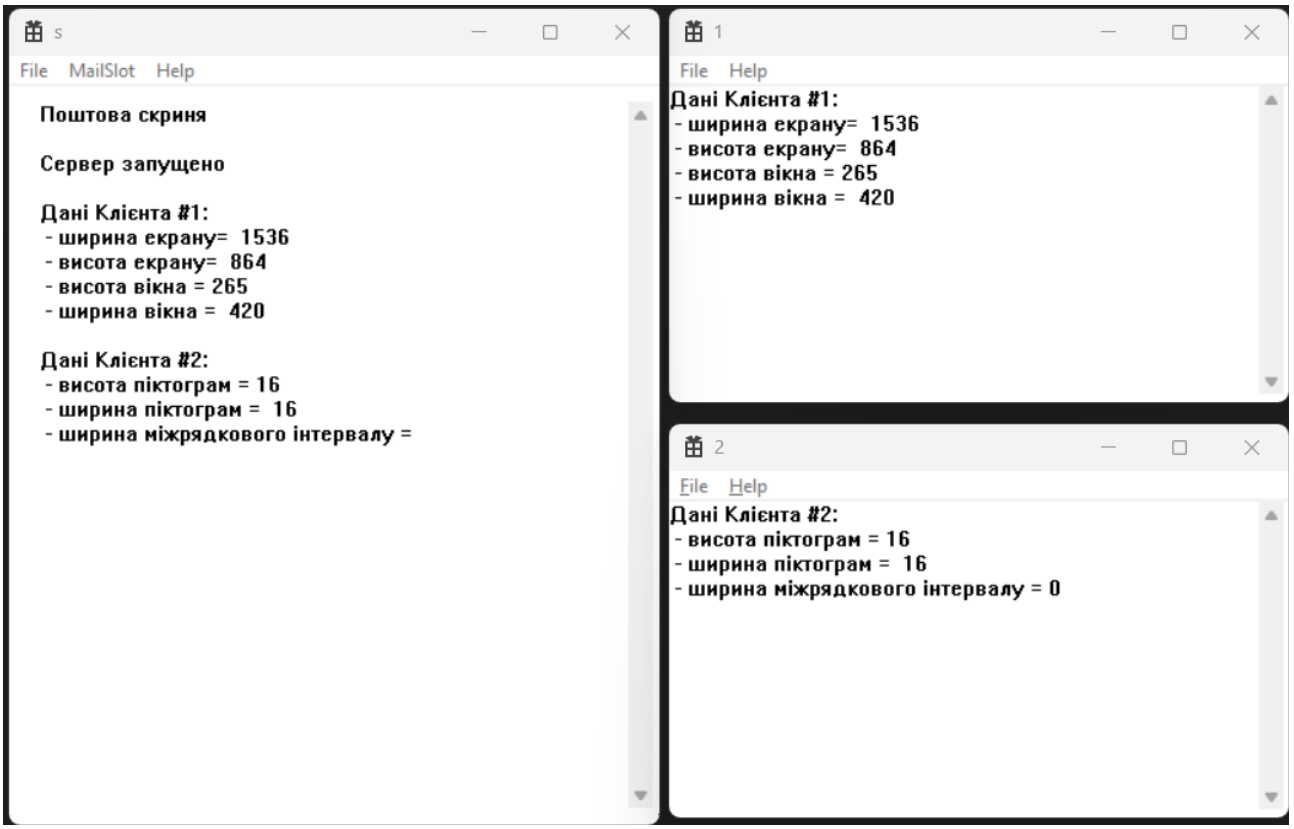


Рисунок 5 – Результат надсилання дефолтного повідомлення від «Клієнт №2»

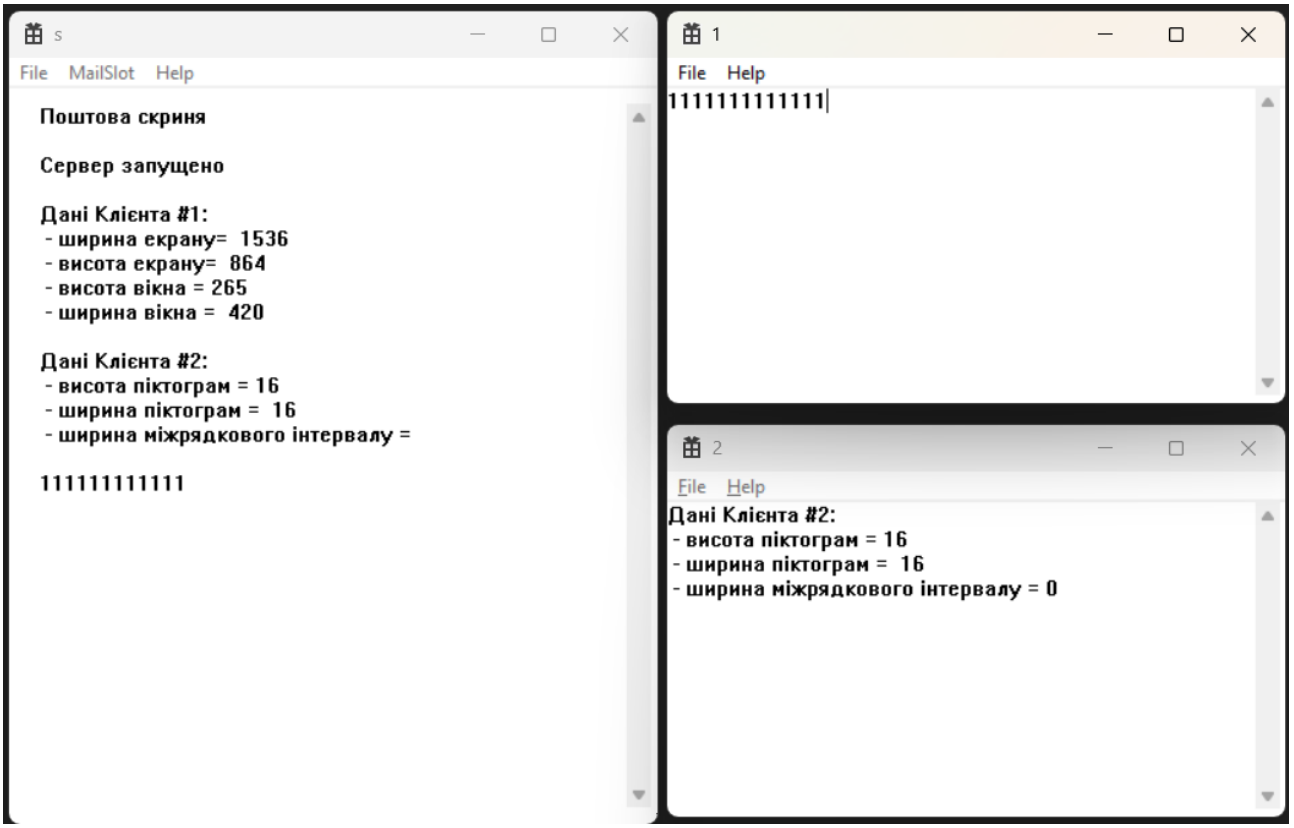


Рисунок 6 – Результат надсилання довільного повідомлення від «Клієнт №1»

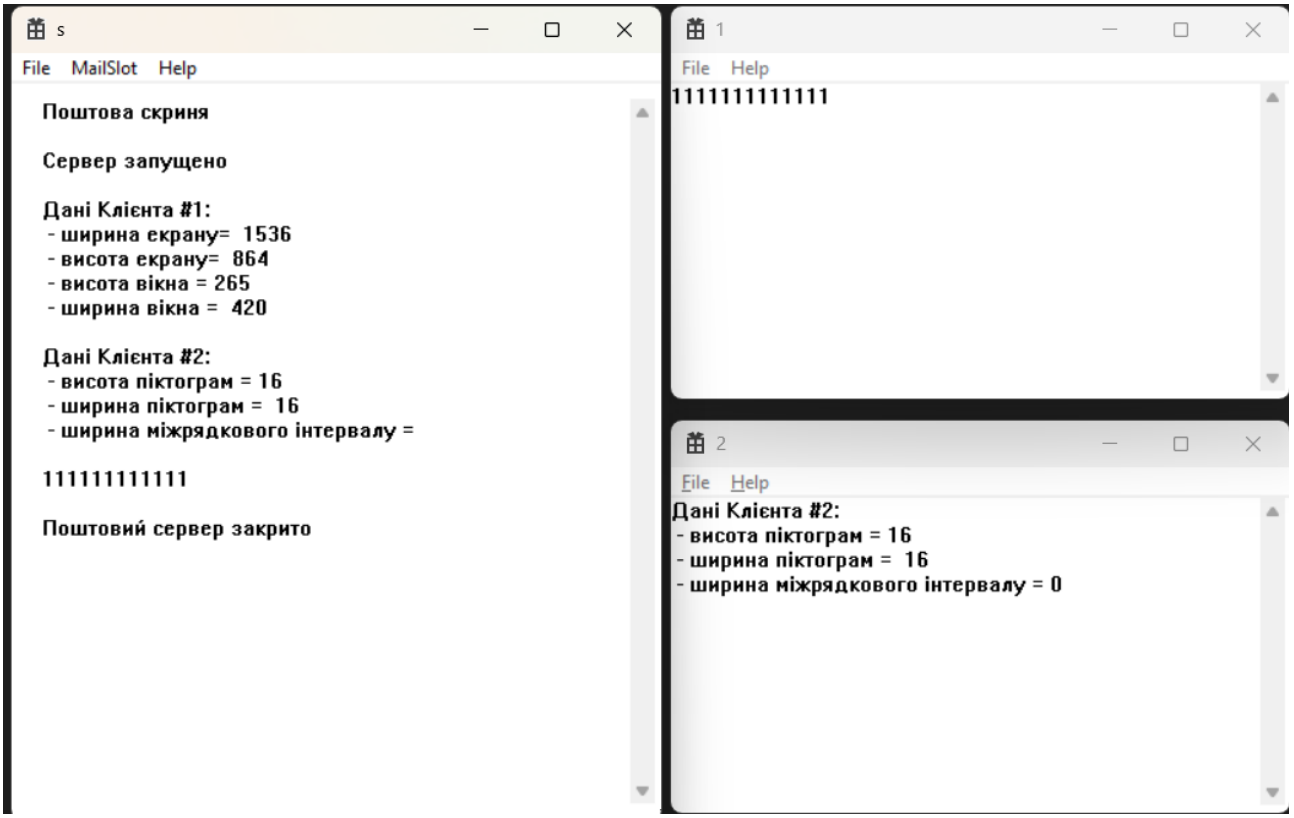


Рисунок 7 – Результат закриття серверу

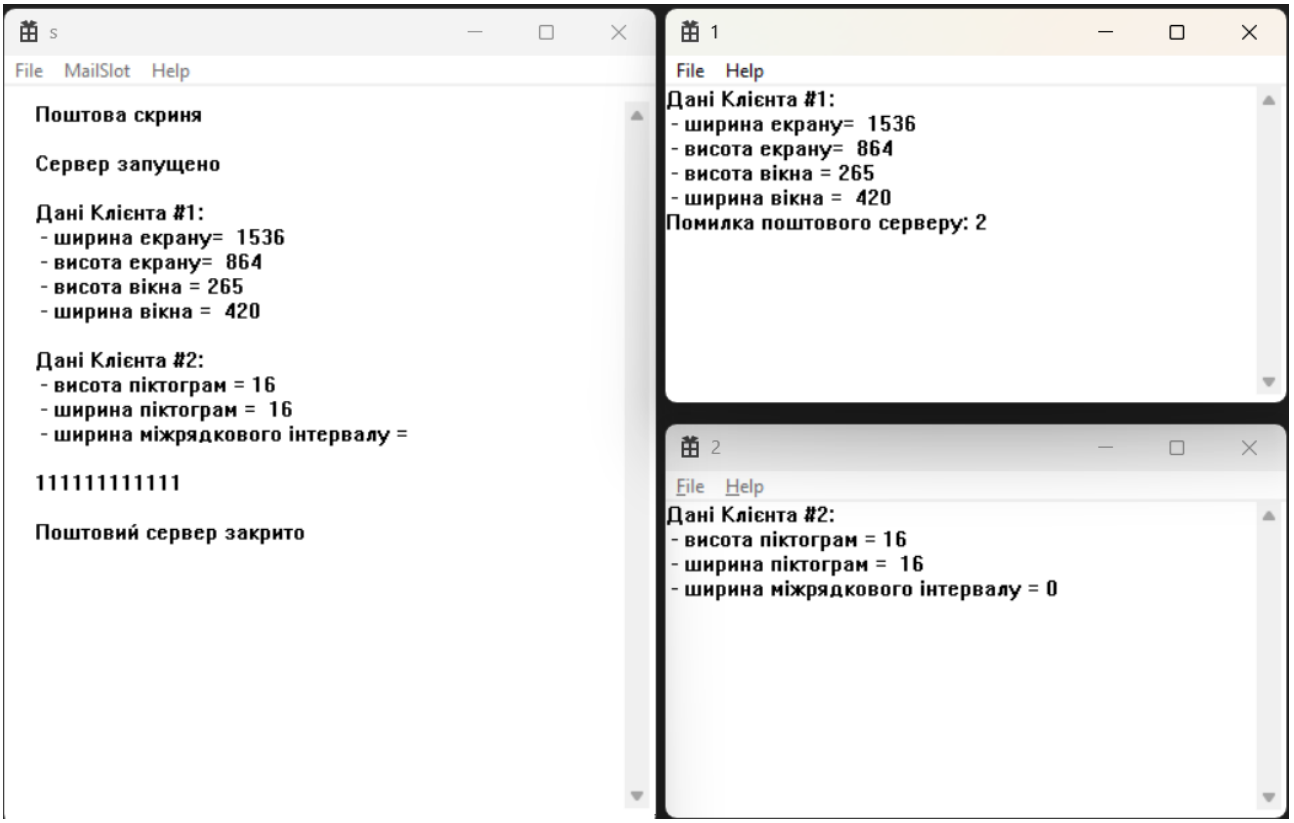


Рисунок 8 – Результат надсилання дефолтного повідомлення від «Клієнт №1» на закритий поштовий сервер

Контрольні запитання

1) Що таке поштова скринька (mailslot) у Windows і для чого вона використовується?

2) Які особливості механізму комунікації за допомогою поштових скриньок порівняно з сокетами?

3) Чим відрізняється «сервер поштових скриньок» від «клієнта поштових скриньок»?

4) Як відбувається створення поштової скриньки у Visual Studio (WinAPI)?

5) Яким чином клієнт надсилає повідомлення у поштову скриньку?

6) Як сервер читає повідомлення з поштової скриньки?

7) Які можливості мережевої взаємодії підтримуються поштовими скриньками (локально, у домені, між вузлами)?

8) Які переваги та недоліки використання поштових скриньок у клієнт-серверних додатках?

9) Чим відрізняється механізм поштових скриньок від буферизації повідомлень у черзі?

10) Які обмеження мають поштові скриньки у Windows (довжина повідомлень, односпрямованість тощо)?

Рекомендована література

1) Шилдт Г. C++: повний курс. – 4-е вид. – К.: Діалектика, 2018. – 1056 с.

2) Microsoft Docs. Mailslots [Електронний ресурс]. – Режим доступу: https://learn.microsoft.com/en-us/windows/win32/ipc/mailslots

3) Richter J. Programming Applications for Microsoft Windows. – Microsoft Press, 1999. – 1696 p.

4) Stevens W. R., Rago S. A. Advanced Programming in the UNIX Environment. – 3rd ed. – Addison-Wesley, 2013. – 1024 p.

5) Petzold Ch. Programming Windows. – 6th ed. – Microsoft Press, 2012. – 1136 p.